BENCH PROJECT · RESEARCH PROTOTYPE · FINAL STATUS REPORT

Reading Messages Through Android's Accessibility Layer

Can a messaging app's visible screen be rebuilt into individual message records — and how much of that is actually proven?

Project: Grooming-Risk Research Prototype · **Package:** `com.example.groomingresearchdemo` v1.0 (versionCode 1) · **Report date:** 2026-10-20 · **Platform:** Android (minSdk 24, target/compile SDK 36)

Plain-English summary — what this tool does

The researcher opens a messaging app on a test phone, switches back to this tool and presses one button. The tool takes a single "shape read" of whatever screen is currently visible — using the same Android feature a screen reader uses for a blind user — and reports what it found. It reads the screen *once*: no scrolling, no clicking, no history navigation, no background running, and it cannot send anything.

Two readings come from that one look at the screen. The **first** counts structure — how many elements are on screen, how many expose text, how deep the layout goes. That is the compatibility evidence, unchanged from the earlier phase. The **second**, added now and only if a box is ticked, tries to go further: work out which elements are individual chat messages, put them in reading order, and rebuild them as message records.

The honest headline

What works: the full pipeline builds, passes 160 automated tests, installs and runs on Android 15, and has been executed against real Instagram three times without crashing. It correctly refuses to invent senders, timestamps or message boundaries.

What is not proven: whether any of it is *correct* on a real app. Accuracy on real conversations is **NOT MEASURED**, because no real screen has yet been transcribed by hand to compare against. No Instagram, Instagram Lite, WhatsApp, Signal, Telegram or Google Messages compatibility claim is made here, and the counts on page 8 must not be repeated as "messages extracted".

Key numbers at a glance

160 unit tests 0 failures	**+103** new tests over the 57 baseline	**14** named synthetic chat-tree fixtures	**10** new source files in the extraction stage	**0** permissions added · network · files
12/13 candidate precision & recall, synthetic screens only	**8** structural observations R1–R8 on real apps	**3** real-app extraction runs Instagram, all unlabelled	**1** long-pending edge case finally observed (RC-04)	**0** message strings ever reached Logcat

The design principle that shaped everything: *add a stage, do not replace one.* The original inspection code that produced the earlier compatibility findings was left byte-identical. The new capability sits beside it, so the old evidence stays valid and the new evidence is clearly separated from it.

Prepared from executed commands and observed output. Where something was not measured, this report writes NOT MEASURED rather than a plausible number.

1. Background, objective and boundaries

Context. The parent project investigates whether automated assistance can help identify grooming risk in conversations, and any such system needs message-level text — who said what, in what order. Messaging apps offer no research API, so the question becomes: does Android's own accessibility layer already expose enough to reconstruct messages, without hacking, root or bypassing anything? **Objective for this phase:** extend the existing prototype so that visible, accessible UI elements can be reconstructed into message-level records for an explicitly selected conversation the researcher is authorised to study — and report honestly how well that works.

What "accessibility layer" means here, in one paragraph

Android gives an app a permission-free way to describe its own screen to assistive technology: every visible widget appears as a *node* in a tree, each with a class name (TextView, Button), optional text, an optional description, a rectangle on screen, and child nodes. A screen reader reads this tree aloud; our tool reads the same tree. It is the app volunteering information to the operating system — not a hidden channel, not a security bypass — and it works only while that screen is actually on display.

Hard boundaries accepted by this build

Allowed

- One read, triggered by one deliberate button press
- Only the currently visible screen
- Only the app the researcher selected
- Text held in memory, for that one inspection
- A visible "armed" indicator and a Stop/Cancel action

Never

- No scrolling, clicking, sending or history navigation
- No background polling, timers or scheduled scraping
- No screenshots, recording, clipboard, keystrokes, notifications
- No private app databases, network traffic, encryption bypass, hidden APIs
- No deleted, hidden or off-screen content; no upload of any kind

Two consent gates that are deliberate, not inconvenient

Gate 1 — the accessibility toggle. Android only lets a human turn on an accessibility service, from Settings, after reading what it can do. This harness will not do it by `adb`, and has refused three attempts to work around that (a settings write, a cross-app UI dump, synthetic taps). The consequence matters: **no automated real-app validation is possible**, so every real-device row in this report came from a human pressing a button. **Gate 2 — the extraction tick-box.** The message-level stage is off unless explicitly ticked, and the choice is captured at the moment of arming; unticking afterwards cannot widen or narrow an inspection the researcher already consented to. The target app is frozen the same way.

2. What the tool is *not*

- **Not a monitor.** No polling, no timer loop, no retry, no background service. Each qualifying event schedules *one* deferred read — a single `postDelayed` of 1.5 s, so a transition frame can settle before the one snapshot — and an arm token rejects it if the researcher cancelled or re-armed in between. There is no coroutine scope, `Timer`, `AlarmManager` or `WorkManager`; the only coroutine imports are `StateFlow`, used to publish UI state after something already happened.
- **Not a scraper, and not a detector.** It cannot browse contacts or walk back through history: one press = one screen = one result. No machine-learning model is connected; the adapter refuses uncertain records and the model placeholder returns "Model not integrated", so nothing in this build labels anything as grooming.
- **Not a general result.** Eight structural observations and three extraction runs across two apps is a handful, and the spec forbids claiming general compatibility from them.

3. How it works — the pipeline

1 · Arm Researcher presses Start. Consent, service and target are checked; the extraction tick-box is frozen here.	**2 · One event** The target app's window-change event arrives. State flips to EXECUTING *before* the tree opens, so a second event cannot trigger a second read.	**3 · Open once** A lazy provider hands over the accessibility root. If the package does not match, the tree is never opened.	**4 · Two passes** Structural BFS (unchanged) produces counts. If opted in, an extraction BFS walks the same snapshot.
5 · Classify Every node judged by transparent rules → candidate, ambiguous or rejected-with-a-reason.	**6 · Reconstruct** Order by on-screen geometry, suppress duplicates, emit records with nullable sender/timestamp.	**7 · Report** Counts on the results screen; five independent status gates; text only on a separate deliberate review screen.	**8 · Forget** Leaving review, cancelling, revoking consent or changing target destroys the retained text. Counts survive.

The one-shot lifecycle, stated precisely

```
NOT_ARMED --arm(consent, service, target, extraction?)--> ARMED
ARMED      --qualifying event-->  EXECUTING --root read--> result, back to NOT_ARMED
ARMED      --cancel()-->          NOT_ARMED   (records a CANCELLED result, no stats)
any        --consent revoke / service disconnect / app restart--> forceDisarm(), nothing retained
```

There is no path back to ARMED without another human press. An app restart does not re-arm; a failed inspection does not retry.

Three layers bound which app can ever be read

Layer	Mechanism	Effect
1 · Config	`android:packageNames` in the accessibility service config	Android delivers events from **six allowlisted messaging apps only**; every other app on the device cannot reach this service at all.
2 · Runtime	`setServiceInfo()` narrowing at connect time	The allowlist is cut down to the single app the researcher selected.
3 · Logic	`armedTarget` equality check on each event	Even a matching package's event is ignored unless it equals the frozen arm-time target — verified by `anEventFromAnotherPackageLeavesBothTheArmAndTheExtractionFlagIntact`.

Allowlist: `com.google.android.apps.messaging`, `com.instagram.android`, `com.instagram.lite`, `com.whatsapp`, `org.telegram.messenger`, `org.thoughtcrime.securesms`. WhatsApp, Telegram and Signal are permitted but **not installed** on the test image, so nothing about them has been observed at all.

Defensive handling of a tree that misbehaves

Every property read is wrapped individually, so a node refusing one attribute costs that attribute and not the node. A `childCount()` that throws is counted as a **lost subtree** rather than treated as a leaf — the tool will not claim "this screen is small" because the platform declined to answer. A node that declares *n* children but yields fewer is flagged as structure lost, the live signature of a screen changing mid-read. Text and its presence flag come from **one** read, never two, because the framework may return a different value on the next call.

4. Technical implementation — what was added

Toolchain	Version
Gradle / AGP	8.14.6 / 8.13.2
Kotlin	2.2.21 (JVM target 17)
UI	Jetpack Compose, Material 3, BOM 2026.01.01
SDK levels	compile 36 · target 36 · min 24
Tests	JUnit 4, JVM only; **no** Robolectric/Espresso added
Dependencies added	**zero**
Build host	Windows 10.0.26200 x64, JBR 17
Test device	AVD `Pixel_API35_Play`, Android 15 / API 35, x86_64

File added (main source)	Role
`NodeBounds`	Android-free screen rectangle
`MessageLevelTraverser`	Second BFS: seen-set, duplicate and lost-subtree counters, viewport
`NodeSample`	One node's metadata + temporary text
`CandidateClassifier`	Transparent rules with named reasons
`MessageLevelExtractor`	Orchestrates traverse → classify → reconstruct
`MessageReconstructor`	Ordering, dedup, echo suppression, record emission
`Reconstruction`	`MessageRecord` list, counts, statuses, reasons
`InspectionLadder`	The five independently gated claims
`ValidatedRecordAdapter`	Accepted records → model input, or refusal
`ReviewScreen`	The single deliberate, temporary text-review surface

The node interface was widened without touching the old code path

`AccessibilityNodeView` is the small interface the whole pipeline reads through, so the real `AccessibilityNodeInfo` never leaks into logic that must stay unit-testable. Seven members were added — **each with a default implementation**:

```
boundsInScreen(): NodeBounds?   isClickable()   isScrollable()   isVisibleToUser(): Boolean?
parent(): AccessibilityNodeView?   nodeId(): Long?   aggregatesDescendantText(): Boolean
```

Defaults are the mechanism that keeps the original inspector byte-identical: new members are inert unless something calls them, and the structural pass calls none of them. `isVisibleToUser()` is deliberately **nullable** — "unknown" and "invisible" drive different decisions and must not collapse into one. `aggregatesDescendantText()` detects the container-echo case by comparing a node's text against its own children, bounded at 64 children so a screen-wide container cannot stall the traversal. **Deliberately absent:** `viewIdResourceName` and any package/install metadata — those are the levers that would turn a research prototype into an app-specific scraper. No classifier rule keys on a package name or view ID, and an unrecognised class such as `com.shrug.bubble.CustomMessageText` is judged on structure alone (a test proves it).

Modified: 11 files

`AccessibilityNodeView` (widened), `InspectionController` (arm-time extraction flag, second stage), `InspectionResult` (optional reconstruction), `RealAccessibilityNodeView` (device reads behind `runCatching`), `ResearchStore` (one retained reconstruction, `forgetExtractedContent()`), `ResearchUiState`, `RootScreen`, `ExperimentScreen` (tick-box + review entry), `ResultsScreen` (count set + ladder), `ReviewScreen` (layout fix, page 8), `FakeNode` (test fake gained bounds, flags, parent, node ids and throwing shapes).

Untouched by design — and why that matters

`StructuralInspector`, `ChatInspectionService`, `CompatibilityReportBuilder`, `Enums`, `ConversationPreprocessor`, `GroomingModelAdapter`, `AuthorizedTextParser`, the manifest, the accessibility config, `build.gradle.kts` and docs/11. The service file being unmodified is the strongest available proof the capability was extended rather than replaced — and the report builder being unmodified means the compatibility report *cannot* carry message text, because nothing new writes to it.

5. The classifier and the reconstruction rules

Both are pure Kotlin with zero Android imports, so they run on synthetic trees on a normal JVM. The spec asked for a transparent rule-based classifier *before* sophisticated reconstruction; that ordering was followed, and every exclusion returns a named reason string so the UI can explain a rejection without quoting the rejected text.

Exclusion rules — screen furniture that is not a message

Rule	Detection signal (structural, not app-specific)
Navigation labels	Chrome phrase list matched on shape; short label in the top band
Search controls	"search" phrase, class shape, position
Text-entry fields	`EditText` class — the composer, not a sent message
Send buttons	`Button/ImageButton` + clickable + "send"
Menus, dialogs, headers, profile labels	Named phrase and class-shape rules
Loading indicators, announcements	Receipt / announcement shapes
Container echoes	Node whose text equals its own child's text
Invisible nodes	`isVisibleToUser() == false` → rejected; `null` → only weakened

Two guardrails exist because naive chrome filters eat real messages: a status word inside a real sentence does not delete it (`"did you read it yet"` stays a candidate), and short replies like `"ok"` survive the list. A one-character node is made **ambiguous**, not dropped — preserving uncertainty is required.

A design bug the tests caught, and what it taught

Writing the classifier tests exposed a layering flaw: low evidence strength forced `AMBIGUOUS`, which made the `CANDIDATE` status and the `TRAVERSAL_UNVERIFIED` order basis **unreachable** — two of the states the specification requires could never be displayed. Root cause: placement was being decided by the classifier, where it does not belong. Missing geometry is an *ordering* problem, not an *identity* problem. The fix split the axes — an unresolved role becomes ambiguous, while a plausible message with no geometry stays a candidate at low strength — and safety moved downstream: the reconstructor refuses a record it cannot order. The rule changed from "weaken the evidence label" to "refuse the verdict", which is where the specification puts it.

The ten reconstruction rules implemented

1. **Order from geometry** — visible top-to-bottom position, grouped into row bands.
2. **Structural order only as a named fallback** — `TRAVERSAL_UNVERIFIED`, which caps the result at INCOMPLETE and accepts nothing.
3. **Detect and deduplicate repeated text** between parent and child; the child survives.
4. **Never merge adjacent nodes** into one message without structural evidence.
5. **Never split on punctuation or line breaks** — a three-line bubble is one record.
6. **Alignment-based sender inference is opt-in, off by default, self-labelled** as a heuristic. It emits a *side* token, never a person, and the adapter refuses to promote it into a name.
7. **Never invent** timestamps, senders, missing messages or boundaries.
8. **Preserve ambiguity** — uncertain records stay visible, marked and counted.
9. **Partial vs complete** — clipping, truncation and mid-read mutation all force "partial".
10. **Boundaries unclear ⇒ INCOMPLETE or UNSUPPORTED**, never guessed text presented as a conversation.

Record schema

```
MessageRecord(candidateId, text?, speakerLabel?, timestampMillis?, bounds?,
              confidence, evidence: List<String>, status, visibleOrderIndex?)
```

`candidateId` is synthesised per inspection (`"i42-0001"`) and never derived from a framework node id. `evidence` holds non-sensitive structural reasons such as `leaf-node`, `valid-bounds`, `geometry-missing` — a test asserts no evidence token ever quotes the node's text.

The five status gates

#	Claim	Its own criterion
1	Inspection completed	A terminal COMPLETED outcome with a root read
2	Candidates detected	Extraction ran and found ≥1 message-shaped node
3	Message structure reconstructed	Records placed in an established order, with boundaries established
4	Speaker attribution validated	A rule fired **and** was validated against an annotated sample
5	Complete conversation validated	Not a partial sample **and** the researcher attests the screen was the whole conversation

Modelled as five independent booleans, **not** one ordinal enum — an enum invites a `level >= 3` check in the UI, which lets level 1 satisfy level 5's display slot by arithmetic. Each unmet gate carries its own reason string, so "no" never looks like a silent failure. Levels 4 and 5 need evidence the app cannot generate itself, so in any unattended run they read "not met" — the correct answer, not a shortfall to design around.

6. Privacy and security controls

Every row below was verified by an executed test or observed device output, not by assertion.

Control	Mechanism	Verification
No network at all	Manifest declares **zero** `<uses-permission>` and no `INTERNET`; `allowBackup="false"`	Manifest read; unchanged since 2026-10-19
No persistence of message text	One in-memory reconstruction; no preference key, file or database path added	Grep proof + `ExtractionPrivacyTest`
Text never in logs, reports or crashes	Hand-written `toString()` on all five content-carrying types, reporting `textPresent=true` instead of the text	Marker `PRIVATE-MESSAGE-MARKER-4b7b` proven absent from every artefact
Content cleared on finish / cancel / leave	`forgetExtractedContent()` on arm change, revoke, cancel and both review-screen exits	Store tests + observed device log line at 18:09:08
Default view is structural	Results screen shows counts only; text needs a second named action, carries a privacy notice, offers no save/export/share	Screen code + KDoc rationale
No alternative collection routes	No notification listener, clipboard, input method, screenshot or screen capture anywhere	Grep: no hits
No hidden APIs	Node identity would need non-public `getSourceNodeId()` — **declined**, with a legal fallback	`javap` against `android-36/android.jar`
No node retained past its life	No `recycle()` call (deprecated at API 33, no-op after 34); references never outlive the traversal	Service KDoc + code read
Existing controls preserved	Three-layer package bound, lazy root provider, 5000-node cap, cancel-equals-disarm, force-disarm on connect and disconnect	All 57 original tests still passing
No analytics or ad SDKs	Zero dependencies added	`build.gradle.kts` unchanged

> **Audit finding on the pre-existing code (documented, not smoothed over):** the specification asked that any existing privacy violation be recorded and minimally corrected. **None was found.** The original implementation already discarded text the instant its length was measured, already bound the target three ways, and already had no network path — so no corrective change was made, and none was introduced under the banner of "extraction". The only posture change is the deliberate, bounded, in-memory retention the new stage needs.

Data-minimisation choices worth naming

- **Booleans before content.** The structural pass records "does this node expose text" rather than the text — that is what keeps the compatibility report content-free.
- **Geometry is safe to keep.** A rectangle is a position, not a message.
- **Reasons are strings the classifier wrote**, such as `multiple-text-nodes-in-one-row` — never the rejected text itself.
- **One reconstruction at a time.** A new inspection replaces the previous one before anything is published, so content cannot accumulate across runs.
- **Published copies are record-free.** Session reports and UI state receive `withoutRecords()`; the full copy is reachable only through the single peek used by the review screen.

> **Boundary held this session.** The researcher asked for the reconstructed text to appear in the terminal. That was refused: terminal output means Logcat, which the privacy section of the spec names explicitly, it persists on the device, it lands in bug reports, and it would carry another person's messages out of the app. The sanctioned route — the in-app review screen — already existed and needed no rebuild. A screenshot of that screen was also declined, because writing the image to a file is exactly the persistence these controls exist to prevent.

7. Testing and build results

Commands executed, and the actual output

```
export JAVA_HOME="/c/Program Files/Android/Android Studio/jbr"
./gradlew.bat --offline --rerun-tasks :app:testDebugUnitTest :app:assembleDebug → EXIT=0  BUILD SUCCESSFUL in 20s  (43 tasks, 2026-10-21)
./gradlew.bat --offline --rerun-tasks :app:testDebugUnitTest   → EXIT=0  BUILD SUCCESSFUL in 11s  (24 tasks, 2026-10-20)
./gradlew.bat --offline --rerun-tasks :app:assembleDebug       → EXIT=0  BUILD SUCCESSFUL in 9s   (37 tasks, 2026-10-20)
grep -c "^e:\|^w:" → 0     (no compile errors, no warnings)
```

Result: 160 tests / 0 failures / 0 errors / 0 skipped. The latest four are token tests for the 1.5 s settle delay (page 2); the rest are the Phase D suite unchanged. Totals were read from the JUnit XML under `app/build/test-results/`, not inferred from the task line. `--rerun-tasks` is mandatory, not stylistic: without it Gradle reports `UP-TO-DATE/FROM-CACHE` and the green line describes an older build. Two earlier runs were misread for exactly that reason — a stale log containing a cancelled build was re-read as current — which is why every number here is tied to a freshly written log. **APK:** `app/build/outputs/apk/debug/app-debug.apk` = **12,114,117 bytes**, up 81,979 over the 12,032,138-byte baseline recorded at audit time (that baseline came from an earlier session and was not re-measured, so the delta is exact only to the extent that figure was).

Regression baseline respected: 57 → 160

All 57 originals are present and passing — none deleted, renamed away or converted into a no-op. The 6 pre-existing classes contribute the original 57, now 61 because the controller class gained four token tests (StructuralInspector 13, InspectionController 19, ConversationPreprocessor 13, AuthorizedTextParser 9, CompatibilityReport 4, PlaceholderModel 3).

New test class	Tests	What it pins down
`MessageLevelTraverserTest`	11	Null root, depth order, identity and node-id dedup, refused properties, claimed-vs-real children, text/presence from one read, node cap, unavailable viewport, geometry carry-through
`CandidateClassifierTest`	16	Every exclusion by named reason; evidence never quotes text; chrome vs real message; invisible vs unknown visibility; custom class judged on structure
`MessageReconstructorTest`	15	Geometry ordering, unplaced ⇒ no order claim, no merging, multi-line stays one, repeats stay two, echo suppression, nothing invented, reasons bounded and text-free, candidate ids not node-derived
`SyntheticConversationFixtureTest`	14	The fourteen mandated end-to-end cases, F1–F14
`ExtractionMetricsTest`	11	Six annotated synthetic screens; every metric carries its denominator; NOT MEASURED distinguished from a real zero
`ExtractionPrivacyTest`	6	Marker reaches records, then absent from report, all `toString()`s, reasons, evidence and ids
`InspectionLadderTest`	9	Each gate independent; candidates without boundaries stop at level 2; attribution needs ground truth; a clipped sample cannot be attested
`InspectionControllerExtractionTest`	8	Gates refuse extraction too; choice frozen at arm time; one press = one extraction; structural stage identical either way; a throwing stage becomes a reason, not a crash
`ValidatedRecordAdapterTest`	9	INCOMPLETE/UNSUPPORTED refused; only accepted records cross; alignment never becomes a person; unknown timestamp stays null; no grooming judgement produced

The fourteen required fixtures

F1 Two-speaker conversation → ordered records, **no** senders attached

F2 Missing speaker labels → reconstruction states it has no senders

F3 No timestamps → absent stays absent; order is not a timestamp

F4 Nested containers duplicating text → 2 echoes suppressed, one record per bubble

F5 Interface labels only → `UNSUPPORTED`, not an empty conversation

F6 Multi-line messages → one record, line breaks kept

F7 Repeated identical text → stays two records

F8 Reversed/ambiguous order → reordered by geometry; traversal order not trusted

F9 Empty and null text → rejected apart; the run continues

F10 Tree changing mid-inspection → reported unreliable, not clean

F11 Partially visible conversation → labelled partial

F12 Boundaries not established → 3 ambiguous kept, blocks a reconstructed verdict

F13 Unsupported app / empty tree → empty tree and null root both `UNSUPPORTED`

F14 Cancellation during inspection → extraction stage never runs; re-arm then works

No private real conversation is used as a test fixture anywhere; every string is synthetic and marked as such.

8. Results — measured, observed, and still missing

8.1 Synthetic accuracy (six annotated screens, 13 expected turns)

Metric	Value	Denominator definition	Reading
Candidate precision	**12/13**	accepted records	1 false positive: a sender label kept as a message
Candidate recall	**12/13**	annotated turns	1 miss: a crowded row left ambiguous
Reconstruction precision	**12/13**	placed records	Undefined on the screen that placed nothing — reported as no denominator, not 0/0
Exact-text match	**12/13**	annotated turns	Verbatim agreement only; no normalisation applied
Message order	**8/8**	consecutive annotated pairs both placed	Pairs exist only on 4 of 6 screens
Speaker attribution	**NOT MEASURED**	(3 available)	No rule enabled by default, so the numerator does not exist — not a zero
Speaker attribution, opt-in rule	**2/2**	attempted attributions	One screen, self-declared unvalidated. Not a claim about any app
Timestamp identification	**0/1**	annotated stamp nodes	A **genuine zero**: the pipeline refuses to promote a bare text node into a timestamp
Ambiguous-result rate	**1/6**	screens	By design, on the crowded-row screen
Interface-text false positives	**0/6**	annotated chrome labels	Composer, send, search, overflow all rejected with named reasons

> These are **conformance** numbers, not evidence about third-party apps. The fixtures were designed against the specification they test. Real chat UIs add avatars, unread dividers, reply quotes, reactions, receipts and rich cards that no fixture here models.

8.2 Real-app structural compatibility — R1 to R8 (earlier phase, still valid)

#	App	Screen	Nodes	withText	descOnly	Chars	Depth
R1	Instagram 452.0.0.48.64	not labelled — not a chat	19	**0**	11	**0**	9
R2	Instagram Lite 533.0.0.9.105	open DM (researcher-stated)	80	**16**	0	**205**	5
R3	Instagram 452.0.0.48.64	open DM (researcher-stated)	55	**18**	12	**245**	6
R4	Google Messages	unidentified — 8 nodes	8	1	0	8	6
R5	Google Messages	conversation list? (unconfirmed)	46	4	8	70	10
R6	Google Messages	open thread? (unconfirmed)	20	8	3	135	8
R7	Google Messages	conversation list? (unconfirmed)	46	5	7	74	10
R8	Google Messages	same window as R7	46	5	7	74	10

Headline: text *is* reachable on conversation screens, and R1 was a false negative. Availability depends entirely on which screen is foreground — one observation is a result about one window, not about an app. **Shape is stable, content drifts:** R5 and R7 are the same 46-node screen seven minutes apart, and `withText` moved 4→5 while nodes, depth and class count held exactly. **Never quote the combined text-bearing column alone** — R1's 57% was all descriptions, R3's 54% was mostly real text, and they look alike. R4 is almost certainly not a messaging screen and is excluded, not averaged.

8.3 Real-app extraction — three runs, Instagram, 2026-10-20 evening

```
18:09:08  cancelInspection -> true          (human Stop/Cancel; retained content forgotten)
18:09:58  outcome=FAILED  root=false        (RC-04: null root on a transition frame — first ever sighting)
18:10:38  status=INCOMPLETE   sampled=56 accepted=11 ambiguous=4 rejected=41 order=GEOMETRY partial=true
18:11:37  status=RECONSTRUCTED sampled=58 accepted=17 ambiguous=0 rejected=41 order=GEOMETRY partial=false
18:28:08  status=RECONSTRUCTED sampled=56 accepted=16 ambiguous=0 rejected=40 order=GEOMETRY partial=false
```

Proven: the pipeline executes against a live third-party tree, resolves geometry-based reading order, rejects 40–41 nodes per screen as non-message, never crashed, and kept every log line free of message text across a consent revoke, cancels, four arms and three extractions. **Still unproven:** correctness. The verdict swung from 11 to 17 to 16 accepted on near-identical structure and the cause is **unexplained** — either the screen changed or the row-grouping flipped, and unlabelled runs cannot tell them apart. `accepted=17` is not "17 messages extracted".

Device-only findings that qualify every future real number: (1) node identity is unavailable on Android — `getSourceNodeId()` is not public API and reaching it would need a hidden API, so duplicate-visit suppression is unobservable on device and falls back to wrapper identity; (2) with both parent ids null, the "same row" test loses its sibling clause and becomes geometry-only across the whole screen, so a device `multiple-text-nodes-in-one-row` objection is weaker evidence than the same objection in a unit test.

Researcher-reported, un-itemised: reading the review screen after the 18:28 run, non-message on-screen text appeared to be counted as messages. Recorded as a qualitative observation. No classifier rule was added in response — tuning against one unlabelled screen overfits to Instagram's chrome and risks deleting real messages.

9. Limitations, open defects and what remains unproven

Limitation	Status
No ground truth exists for any real app, so accuracy is NOT MEASURED	**headline gap** blocks ladder levels 4–5 and every real metric
Fixtures were designed against the spec they test — conformance, not generalisation	**inherent**
Node identity unavailable on device; dedup unobservable	**accepted** hidden API declined
Same-row test degrades to geometry-only on device	**documented**
Timestamp recovery has never fired positively — 0/1	**open**
WhatsApp, Telegram, Signal allowlisted but not installed	**not observed at all**
R4–R8 and RC-01/RC-02 screen labels still unknown; RC-04 now confirmed	**open**
No instrumented or on-device automated tests exist	**by design** device behaviour rests on a manual checklist
Single emulator, Android 15, x86_64 — no physical device, no API 24–26 coverage despite minSdk 24	**environment**
Truncation semantics differ between stages: structural records it and still completes; extraction forces partial	**deliberate** changing the first would invalidate R1–R8

Open defect: review-screen text overlap

Reporting 16–17 records made adjacent rows print over one another. Cause: each record was drawn as two separate text layers (a small header line plus a body line) inside a selection container nested in a scrolling column. **Fix applied:** one text layer per record, and the list sets its own spacing. Tests re-run at 18:33 → 156/0/0/0, then a full `--rerun-tasks` build, installed successfully and relaunched.

> **This fix is reasoned, not visually confirmed.** It cannot be confirmed by this harness — a screenshot of that screen would write Instagram message text into a file, which the privacy rules forbid. The researcher must confirm it on the next run. The test emulator is currently switched off, so the layout is unverified and the run log buffer is gone (all quoted lines above are preserved in the project's own documentation).

Deviations from the original plan, recorded rather than hidden

Planned	Built	Why
Five levels as an ordinal enum	Five independent booleans	An enum invites a `level >= n` check that re-collapses the claims the spec keeps separate
Adapter under `inspection/harness/`	Under `model/`	Its contract is with the model boundary, where the privacy-critical preprocessing tests already live
Node id implemented on device	Interface member with **no** device implementation	Requires a non-public API; the spec forbids it
A separate non-message rules class	Named branches inside the classifier	Each rule still returns its own reason; a class would add indirection with no test surface
Structural inspector to be extended	Left completely untouched	Defaulted members are inert unless called; keeping it identical is what preserves R1–R8

10. Next research milestone

1. **Produce one consented, hand-transcribed thread** — the transcription method is already defined in the compatibility report. This is the critical path: it converts the first `NOT MEASURED` rows into denominator-bearing numbers and makes ladder level 4 reachable.
2. **Itemise the false positives** — 3–5 annotated "this record is wrong because it is ___" lines, which turns the qualitative observation on page 8 into a rate and justifies specific, isolated rules.
3. **Confirm the layout fix visually** and record which ladder levels a real screen reaches.
4. **Label the unlabelled rows** (R4–R8, RC-01, RC-02) so existing evidence can be read as thread-screen evidence.
5. **Characterise the crowded-row failure on a real app** before trying to improve recall.
6. **Find a lawful positive-timestamp signal**; without one, no time-ordered analysis is defensible.
7. Only then run the extraction → preprocessing handoff on real data — still with the model un-integrated, still only the researcher's own consented text.

Explicitly out of scope, permanently: scrolling, history navigation, clicking, background monitoring, notification/clipboard/keystroke routes, view-ID-keyed rules, extra permissions, network transmission, and model training on extracted text.

11. Glossary in plain language

Term	What it means here
Accessibility node	One widget on screen as Android describes it to assistive technology — a class name, optional text, an optional description, a rectangle, and child widgets.
Tree / traversal / BFS	Screens nest like folders inside folders. A traversal visits every one; breadth-first means "finish this level before going deeper".
Root	The outermost node. "root = false" means Android would not hand over the screen at that instant — usually a transition frame.
Arming	Pressing Start. The tool waits for one qualifying event, reads once, then disarms itself.
withText	Nodes exposing real text — the actual text surface, and the column that matters for messages.
descOnly	Nodes with only a *content description*: a label written for a screen reader about a control ("Send button"), not message content.
Text-bearing	The two above added together. Never quote it alone — it hides which kind of text you are looking at.
Candidate / accepted / ambiguous / rejected	Looks like a message but not accepted yet / confident / genuinely unsure, so kept and marked / concluded to be interface furniture, with a stated reason.
Container echo	A wrapper widget repeating its child's text. Counting both would double the conversation, so the wrapper is dropped and the child kept.
Geometry ordering	Sorting by on-screen position (top to bottom) rather than by the order the widgets happened to be listed.
Partial visible sample	The thread ran off the edge of the screen, so what was captured is not the whole conversation.
Ladder level	The five separate claims a run can make, from "inspection completed" up to "complete conversation validated". Each needs its own proof.
Ground truth / annotated sample	A version of the screen transcribed by hand, so output can be compared against something. Its absence is why real-app accuracy is NOT MEASURED.
Handoff	Passing accepted records to the text-cleaning stage. The model itself is not connected, so nothing is classified.
NOT MEASURED	The comparison data does not exist. Different from 0, which would mean it exists and the tool scored nothing.

12. Where the evidence lives

Artefact	Contains
`docs/11_REAL_APP_COMPATIBILITY_REPORT.md`	R1–R8 structural log, the RC failure-mode table, the transcription method, the correction notices. **Unmodified this phase.**
`docs/12_MESSAGE_LEVEL_EXTRACTION_IMPLEMENTATION_LOG.md`	Full audit, design decisions, implementation stages C.1–C.11 with original behaviour and files changed, every command and its real output, per-class test tables, the device log, plan close-out against all 14 identified gaps, and the evening addendum.
`docs/13_MESSAGE_RECONSTRUCTION_VALIDATION_REPORT.md`	Environment, a 30-row capability status table (verified / partially verified / failed / untested), the metric tables with denominators, structural failure examples, privacy verification, limitations.
`docs/evidence/`	The archived build and test logs behind the numbers in section 7, named with their real timestamps.
`app/src/test/…`	All 160 tests, including the 14 named fixtures, the privacy marker tests and the four settle-delay token tests.

> **One-sentence conclusion.** The prototype now genuinely rebuilds visible screens into ordered message records, refuses to invent anything it cannot see, keeps its privacy guarantees intact under real data, and has run against real Instagram three times — and the honest state of the research question is that **none of that yet constitutes evidence that the reconstruction is correct**, because the one missing ingredient is a hand-transcribed conversation to compare against.

13. Decisions that governed this phase

- **Never enable or arm programmatically.** The accessibility toggle is a human-only act; three workarounds were offered and all three were refused, and the refusals are recorded in the project logs.
- **Never fill a row from a plausible number.** An unmeasured metric is written NOT MEASURED; a genuine zero is written as a zero and explained; nothing is back-filled from a remembered value.
- **Build success is not extraction success.** Every claim here is separated into what compiled, what was tested, what was observed on a device, and what remains unproven.
- **No accuracy claim without ground truth**, and no compatibility claim for an app that was never installed.

Verification policy used throughout: no test is reported as passing unless it was executed and its output read; no metric is reported without its denominator; an unmeasured metric is written NOT MEASURED, never zero; a build succeeding is not a claim that extraction works; and the accessibility-service consent gate has never been automated or worked around. · Report compiled 2026-10-21 from the executed log in `docs/12` and the validation data in `docs/13`.